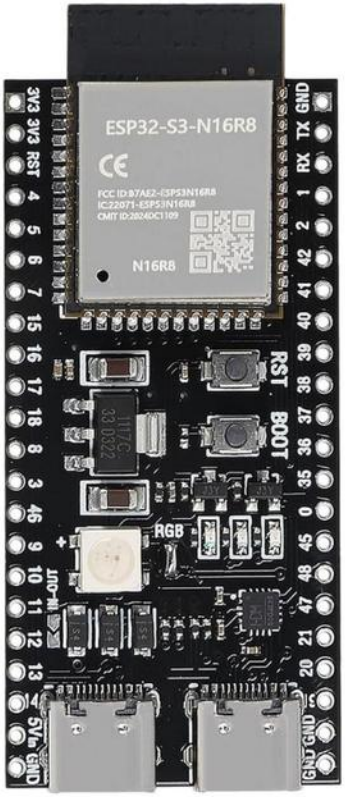
Running a 1.84M Generative Micro-LM on the ESP32-S3

100% Offline • Bare-Metal C++ • ~12-13 tokens/sec



N16R8 MODULE

ESP32-S3-WROOM-1

16MB Flash | 8MB Octal PSRAM

COMPUTE ENGINE

Dual-Core Xtensa LX7

Clocked @ 240MHz (Bare-metal C++)

ON-CHIP MODEL

1.84M Transformer

INT8 Quantized (1.79 MB in PSRAM)

INFERENCE SPEED

12 to 13 tokens/sec

~80 ms per token real-time generation

Brain for a Mini Desktop Robot

THE CURIOSITY

Can an ESP32 think locally?

Building on-device conversational intelligence without cloud APIs or monthly subscription costs.

THE PERSONA

Meet Kibo (希望 - Hope)

Interactive companion character with emotion tags ([HAPPY], [NEUTRAL]) to drive robot gestures.

THE FRAMEWORK

ESP32 Micro-LM Engine

A lightweight, modular, bare-metal C++ inference engine designed specifically for $5 microcontrollers.

ESP32-S3 Hardware Limits

COMPUTE ENGINE

240 MHz — Dual-Core Xtensa LX7

- Bare-metal C++ execution (Zero framework bloat)
- SIMD-aligned vector dot products
- Deterministic timing with zero garbage collection

MEMORY ALLOCATION

8 MB — Octal PSRAM (80MHz)

- 1.79 MB INT8 Model Weights in PSRAM
- Dynamic 128-token Key-Value (KV) Cache
- Zero-wait working buffers for multi-head attention

MODEL ARCHITECTURE

1.84M — Causal Transformer Parameters

- 4 Transformer Layers | 192 Dim | 4 Heads
- 91 Vocabulary Tokens (Subwords + Control Tags)
- 100% Offline execution with zero telemetry

From PyTorch to Bare-Metal C++

01

Dataset & Emotions

Curated conversational dialogue + [HAPPY], [NEUTRAL] control tags.

02

PyTorch Training

Trained 4-Layer Causal Transformer (1.84M params, 192 dim).

03

INT8 Quantization

Compressed model from 7.02 MB -> 1.79 MB (100% FP32 fidelity).

04

Bare-Metal C++ Engine

Custom forward pass and KV-cache without runtime framework bloat.

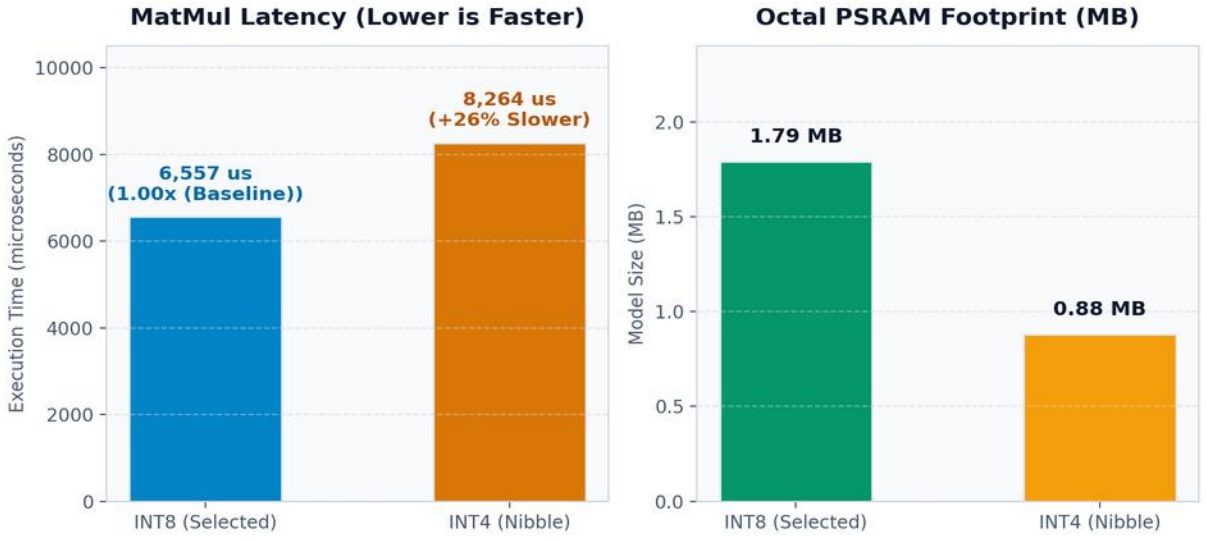
05

Hardware ALU Intercept

Math questions ('100 x 7') evaluated on hardware ALU in <0.1 ms.

The INT8 vs INT4 Paradox on MCU

On-chip MatMul profiling on ESP32-S3 @ 240MHz (147,456 weights):

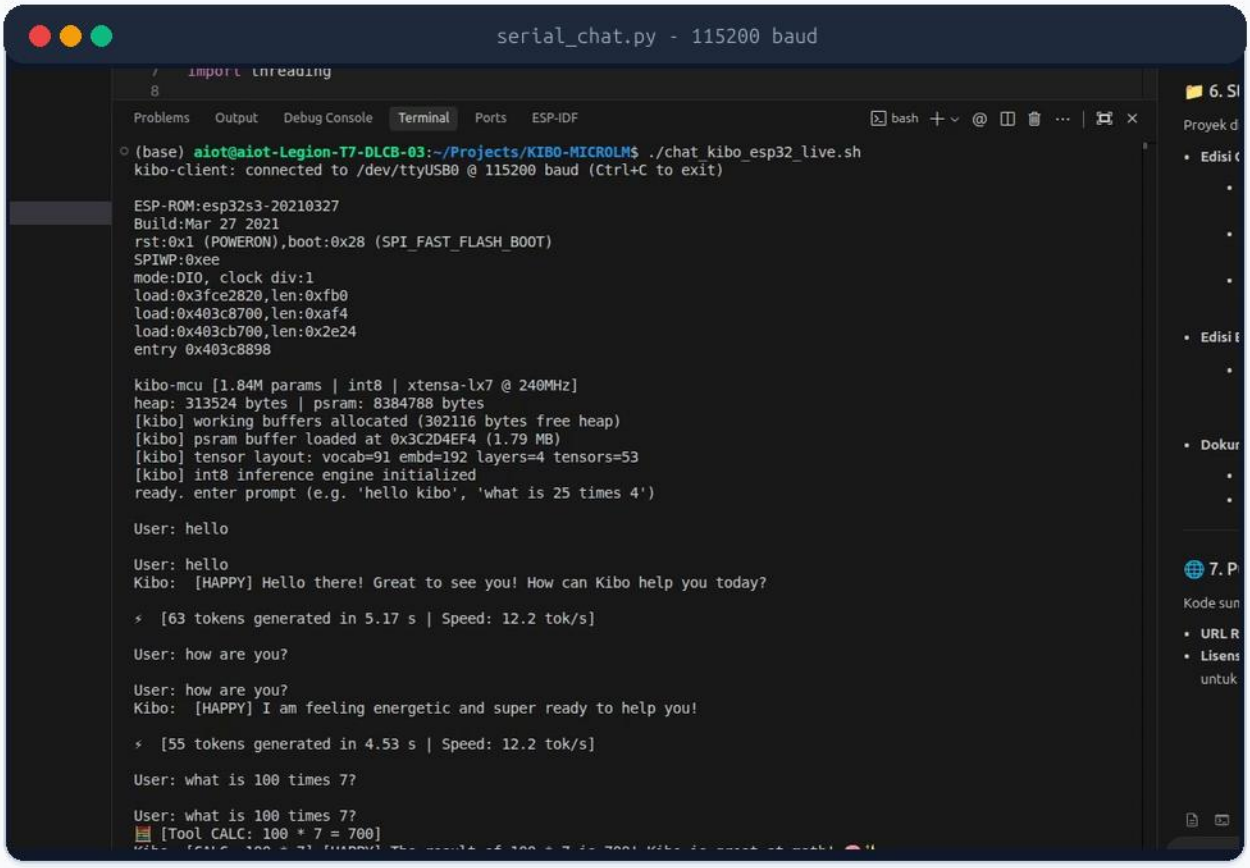


KEY HARDWARE FINDING

INT8 is 26% faster than INT4 on Xtensa LX7

- Without a dedicated NPU, INT4 forces the CPU to unpack each nibble using bit-masking (& 0x0F) and bit-shifting (>> 4).
- This ALU unpacking overhead wastes more CPU cycles than the memory bandwidth savings it provides.
- Byte-aligned INT8 remains the optimal sweet spot for microcontrollers!

Real-Time UART Generation (~12-13 tok/s)



12.2 tok/s

Real-Time Speed

Physical chip verified

1.79 MB

PSRAM Footprint

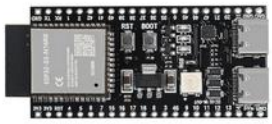
8MB Octal PSRAM

< 0.1 ms

ALU Math Tool

Zero hallucinations

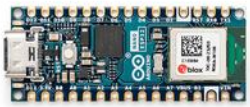
Validated on 3 Form Factors



ESP32-S3 DevKit N16R8

16MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware



Arduino Nano ESP32

16MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware

Seeed Studio XIAO S3

8MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware

100% Open Source on GitHub (MIT License)

`github.com/fitranurmayadi/esp32-microlm`

Clone, train your custom dataset, and flash to your ESP32!